

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

BHUBANESWAR | ODISHA

SMART CAMPUS NAVIGATION AND ROUTE OPTIMIZATION SYSTEM

A Graph-Based C++ Object-Oriented Software Implementation

SUBMITTED BY	SUBMITTED TO
Preetika Mishra Student ID: B125086 Branch: CSE-B Semester: 3rd Semester	Dr. Srichandan Sobhanayak Department of Computer Science & Engineering

OBJECT-ORIENTED PROGRAMMING (OOPS) PROJECT

ACADEMIC SESSION 2026–27

CERTIFICATE

This is to certify that the project report entitled “Smart Campus Navigation and Route Optimization System” has been prepared and submitted by Preetika Mishra (Student ID: B125086), CSE-B, 3rd Semester, as part of the Object-Oriented Programming (OOPS) Project for the academic session 2026–27.

The report presents the design, implementation and source-based analysis of a C++ console application that models campus locations and connecting pathways, computes routes using Dijkstra’s algorithm, and supports administrative pathway management with file-based persistence. The certificate is provided for review and signature by the supervisor.

Project Supervisor	Student
Dr. Srichandan Sobhanayak Signature: _____ Date: _____	Preetika Mishra Signature: _____ Date: _____

Department of Computer Science & Engineering

International Institute of Information Technology, Bhubaneswar

DECLARATION

I, Preetika Mishra (Student ID: B125086), student of CSE-B, 3rd Semester, hereby declare that the project report titled “Smart Campus Navigation and Route Optimization System” has been prepared for the Object-Oriented Programming (OOPS) Project during the academic session 2026–27.

The technical description in this report has been aligned with the C++ source code and data files provided for the project. Where the earlier report and the implementation differ, the implementation has been treated as the authority for claims about program behaviour. Calculations presented as examples are identified as derived calculations rather than recorded execution logs.

I understand that the report should be reviewed for academic correctness and that the declaration is to be signed by me at submission.

Student Name: Preetika Mishra

Student ID: B125086

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Srichandan Sobhanayak for guidance and academic support during the development of this Object-Oriented Programming project. The project provided an opportunity to apply C++ classes, data encapsulation, graph modelling, algorithmic problem solving and file handling to a practical campus-navigation problem.

I also acknowledge the learning resources and standard references that support the study of C++ programming and shortest-path algorithms. Finally, I thank the institute and the academic environment at IIIT Bhubaneswar for providing the context in which this project was conceived.

The report has been prepared with reference to the supplied project report and the accompanying source archive. Particular care has been taken to describe the code that is present, and to separate current functionality from possible future work.

ABSTRACT

The Smart Campus Navigation and Route Optimization System is a C++17 console application that represents a logical model of the IIIT Bhubaneswar campus as an undirected weighted graph. Campus facilities are represented by Location objects, while connections between facilities are represented by Path objects carrying a physical distance and an availability state. The supplied dataset contains 22 locations and 31 initial pathways.

The Campus class coordinates directory operations, route calculation, pathway administration and persistence. Route calculation uses Dijkstra's single-source shortest-path method. In shortest-distance mode, edge cost is the pathway length in metres. In fastest-estimated-time mode, each edge is converted to an estimated traversal time using assumed speeds of 1.2 m/s for pathways shorter than 150 m and 1.5 m/s for pathways of 150 m or more. Blocked pathways are skipped during route relaxation.

Administrators can add, edit and remove connections, and block or restore existing connections. The graph structure is stored in paths.txt; blockage state and reasons are stored in path_status.txt. The constructor loads the location catalogue, paths and saved blockages. The interface is terminal-based; GPS, live traffic sensing, GUI rendering and cloud synchronisation are not implemented.

Implementation note The supplied path_status.txt is empty at inspection time, so the packaged starting state contains no persisted closures. The report's rerouting example is a scenario derived from the graph and code, not a claim that a saved blocked route was present in the archive.

TABLE OF CONTENTS

CERTIFICATE 2
DECLARATION 3
ACKNOWLEDGEMENT 4
ABSTRACT 5
TABLE OF CONTENTS 6
LIST OF FIGURES AND TABLES 8
ABBREVIATIONS 8
1. INTRODUCTION 9
1.1 Background and institutional setting 9
1.2 Project overview 9
1.3 Problem statement 9
1.4 Project motivation 9
2. REQUIREMENTS, MOTIVATION AND SCOPE 10
2.1 Functional requirements 10
2.2 Non-functional considerations 10
2.3 Scope and boundaries 10
2.4 Existing approach and proposed system 10
3. SYSTEM ARCHITECTURE AND COMPONENT DESIGN 11
3.1 Component responsibilities 11
3.2 Startup sequence 11
3.3 Data flow 11
4. OBJECT-ORIENTED DESIGN AND CONCEPTS 12
4.1 Encapsulation 12
4.2 Abstraction 12
4.3 Composition / has-a relationship 12
4.4 Constructors, const correctness and operator overloading 12
4.5 Concepts not demonstrated 12
5. CLASS RESPONSIBILITIES AND IMPLEMENTATION 13
5.1 Location 13
5.2 Path 13
5.3 Campus 13
6. CAMPUS GRAPH DATASET: LOCATIONS 14
7. CAMPUS GRAPH DATASET: PATHWAYS 15
8. ROUTE OPTIMIZATION USING DIJKSTRA'S ALGORITHM 17
8.1 Cost model 17
8.2 Algorithm steps 17
8.3 Verified implementation excerpt 17
8.4 Output interpretation 17
9. PATH MANAGEMENT AND DYNAMIC RE-ROUTING 18

9.1 Administrative operations....18
9.2 Re-routing example: Hostel E to Open Recreation Ground....18
10. FILE PERSISTENCE AND PROGRAM WORKFLOW....19
10.1 File formats....19
10.2 Loading and saving....19
10.3 Main program workflow....19
10.4 Input validation and admin gate....19
11. IMPLEMENTATION, TESTING AND COMPLEXITY....20
11.1 Source layout and build....20
11.2 Testing evidence and limits....20
11.3 Complexity analysis....20
12. LIMITATIONS, FUTURE WORK AND CONCLUSION....21
12.1 Current limitations....21
12.2 Future enhancements (proposed, not implemented)....21
12.3 Conclusion....21
REFERENCES....22
APPENDIX A. VIVA-READY TECHNICAL SUMMARY....22
APPENDIX B. SOURCE VERIFICATION NOTES....22

LIST OF FIGURES AND TABLES

Item	Title
Figure 1	System component relationship (console → Campus → models and files)
Figure 2	Startup and persistence sequence
Figure 3	Route calculation and path reconstruction flow
Figure 4	Hostel E to Recreation Ground rerouting scenario
Table 1	Functional and non-functional requirements
Table 2	Class responsibilities and public operations
Table 3	Campus location vertices (22 records)
Table 4	Initial pathway edges (31 records)
Table 5	Test scenarios and evidence status
Table 6	Complexity of principal operations

ABBREVIATIONS

Term	Meaning
CLI	Command-line interface
OOP	Object-Oriented Programming
SSSP	Single-Source Shortest Path
STL	C++ Standard Template Library
ID	Location identifier

1. INTRODUCTION

1.1 Background and institutional setting

A campus contains multiple destinations—teaching spaces, laboratories, administrative offices, accommodation, dining facilities and recreation areas. For a first-time visitor or a student unfamiliar with a particular part of the campus, knowing the destination name is not always enough; the user also needs a sequence of connected walkways. A temporary closure can make a familiar route unavailable.

1.2 Project overview

This project translates a logical campus layout into a graph. A Location is a vertex with an integer ID, name, category and description. A Path is an undirected connection between two location IDs with a positive distance, a blocked flag and an optional closure reason. Campus owns vectors of these objects and provides operations for searching, displaying and modifying the model.

1.3 Problem statement

Given a source and destination among the configured locations, the application must find a minimum-cost route that uses only currently open pathways. The cost criterion is selected by the user: physical distance or estimated traversal time. If no open route connects the two endpoints, the application reports that no available route exists.

1.4 Project motivation

The project combines class-based modelling with a classical graph algorithm and persistent data updates. It is intended as an educational software implementation, not as a surveyed or GPS-accurate navigation service.

2. REQUIREMENTS, MOTIVATION AND SCOPE

2.1 Functional requirements

Requirement	Implemented behaviour verified in source
Campus directory	Displays location IDs, names and categories.
Location search	Search by ID or case-insensitive name substring.
Logical map	Prints a fixed text schematic and explicitly labels it logical, not GPS geometry.
Route selection	Offers shortest-distance and fastest-estimated-time modes.
Path display	Lists connections and can filter open or blocked paths.
Administration	Hard-coded login gate; add, edit, remove, block and restore connections.
Persistence	Reads/writes paths.txt and path_status.txt.

2.2 Non-functional considerations

- Input robustness: integer input is parsed from a line; invalid values are rejected and re-prompted.
- Data validation: path endpoints must exist and differ; distances must be finite and positive; duplicate undirected connections are rejected.
- Maintainability: declarations and definitions are separated into headers and source files.
- Persistence safety: several failed-save paths attempt to roll back the in-memory change.

2.3 Scope and boundaries

The delivered program is a single-user terminal application. It has no graphical map, GPS positioning, live traffic feed, mobile client, network service or database. The map printed by displayMap() is a conceptual schematic rather than a geometrically accurate campus plan.

2.4 Existing approach and proposed system

Static signboards and printed maps provide orientation but do not compute an optimal path or automatically remove a temporarily closed connection from a route calculation. This program improves on that limited workflow by computing routes from a mutable graph and saving administrator changes locally. The comparison is conceptual; no measured comparison study is included in the source archive.

3. SYSTEM ARCHITECTURE AND COMPONENT DESIGN

The source is modular, but it is more accurate to describe it as a console front end, a central Campus manager, two domain model classes and two local text files—not as a separately deployed multi-tier service.

CONSOLE / PRESENTATION	CAMPUS MANAGER	DOMAIN MODEL + STORAGE
main.cpp Menus, prompts, readInt(), admin login	Campus.h / Campus.cpp Directory, route solver, path operations, load/save	Location.h/.cpp — vertex metadata Path.h/.cpp — weighted connection paths.txt — edge records path_status.txt — closures

3.1 Component responsibilities

- main.cpp reads user choices and calls Campus public methods. It does not directly edit the private vectors.
- Campus stores std::vector<Location> and std::vector<Path>, validates path operations, performs Dijkstra's algorithm and synchronizes files.
- Location stores descriptive metadata for a campus place.
- Path stores the two endpoint IDs, distance, blocked state and closure reason.

3.2 Startup sequence

```
main()
  └─ Campus campus;
       ├─ loadLocations()      // fixed catalogue in Campus.cpp
       ├─ loadPaths()          // parse paths.txt
       └─ loadPathStatuses()   // apply saved closures, if any
```

The Campus constructor invokes these three operations in this order. A missing path_status.txt is treated as no saved blockages. A missing paths.txt produces an error message and leaves the paths collection empty.

3.3 Data flow

A route request supplies two IDs and a mode flag to Campus::findRoute(). The manager validates the IDs, scans open Path objects, updates tentative costs and predecessor IDs, reconstructs the route, then prints the location sequence, physical distance and estimated time.

4. OBJECT-ORIENTED DESIGN AND CONCEPTS

4.1 Encapsulation

Location, Path and Campus declare their data members private. External code accesses location details through const-qualified getters and displayDetails(); Path exposes getters and controlled setters; Campus exposes operations instead of its vectors. This protects internal representation from direct modification by main.cpp.

4.2 Abstraction

The menu presents operations such as “Find Route”, “Block a path” and “Restore a path”. The caller does not need to know how Campus::findRoute() selects vertices or how savePathStatuses() serializes closure records. These public operations provide a higher-level interface over implementation details.

4.3 Composition / has-a relationship

Campus contains std::vector<Location> and std::vector<Path> as value members. This is a concrete has-a relationship: Campus manages collections of the model objects. The vectors own their stored values and manage their dynamic storage through the C++ standard library.

4.4 Constructors, const correctness and operator overloading

Location and Path have constructors that initialize their fields. Accessor methods are const-qualified where appropriate. Location additionally declares a friend stream insertion operator (operator<<), implemented in Location.cpp, and displayDetails() uses it to print the object.

4.5 Concepts not demonstrated

Accuracy boundary The inspected implementation does not demonstrate inheritance, virtual functions or runtime polymorphism, nor does it use a custom class hierarchy. The README lists broader OOP topics, but those topics are not supported by the actual class declarations and source implementation.

The project does use compile-time operator overloading through the friend operator<< for Location. There is no evidence in the inspected code of function overloading as a design feature, custom static members or manual dynamic-memory ownership.

5. CLASS RESPONSIBILITIES AND IMPLEMENTATION

Class	Private state	Main public interface / responsibility
Location	int id; string name, category, description	Constructor; getId(), getName(), getCategory(), getDescription(); displayDetails(); friend operator<<. Models one campus vertex.
Path	int startId, endId; double distance; bool blocked; string blockReason	Endpoint/distance getters; setDistance(); isBlocked(); setBlocked(); getBlockReason(); getOtherEnd(). Models one undirected edge.
Campus	vector<Location> locations; vector<Path> paths	Directory, map, statistics, search, path listing, add/edit/remove/block/restore, findRoute. Coordinates the graph and persistence.

5.1 Location

```
class Location {
private:
    int id;
    std::string name, category, description;
public:
    int getId() const;
    std::string getName() const;
    void displayDetails() const;
    friend std::ostream& operator<<(std::ostream&, const Location&);
};
```

The Location constructor uses an initializer list and std::move for the string parameters. The class is a data model and does not contain routing logic.

5.2 Path

```
int Path::getOtherEnd(int locationId) const
{
    if (locationId == startId) return endId;
    if (locationId == endId) return startId;
    return -1;
}
```

getOtherEnd() makes the undirected nature of a pathway explicit. It returns the opposite endpoint when the supplied ID is incident to the path and -1 otherwise.

5.3 Campus

Campus is the coordinating class. Its private helper findLocation() performs a linear search and returns a pointer to a matching Location or nullptr. The static helper toLower() normalizes a copied string for case-insensitive name search.

6. CAMPUS GRAPH DATASET: LOCATIONS

The fixed catalogue is populated by Campus::loadLocations() in Campus.cpp. The following names, IDs, categories and descriptions are drawn from that implementation. IDs are contiguous from 1 through 22, which the route solver relies on when sizing and indexing its working vectors.

ID	Location	Category	Description
1	Main Gate	Entrance	Primary entrance and exit of the IIIT Bhubaneswar campus.
2	Academic Block	Academic	Main academic area containing classrooms and teaching spaces.
3	Central Library	Academic	Library containing books, journals and digital academic resources.
4	Laboratory Complex	Academic	Academic laboratory area used for practical and technical work.
5	Administrative Block	Administration	Administrative offices and institutional services.
6	Auditorium	Events	Venue for seminars, cultural programmes and institute events.
7	Guest House	Accommodation	Guest accommodation facility within the campus.
8	College Canteen	Food	Campus dining and refreshment facility.
9	Sports Complex	Sports	Sports and recreational facilities for students.
10	Shopping Area	Utility	Campus shopping and essential student services area.
11	Hostel D	Boys Hostel	Residential hostel block D.
12	Hostel E	Boys Hostel	Residential hostel block E.
13	Hostel F	Boys Hostel	Residential hostel block F.
14	Hostel G	Boys Hostel	Residential hostel block G.
15	Hostel I	Boys Hostel	Residential hostel block I.
16	Hostel J	Boys Hostel	Residential hostel block J.
17	Hostel K	Girls Hostel	Residential hostel block K.
18	Hostel L	Girls Hostel	Residential hostel block L.
19	Hostel P	Girls Hostel	Residential hostel block P.
20	Hostel Q	Girls Hostel	Residential hostel block Q.

ID	Location	Category	Description
21	Main Parking Area	Transport	Parking and vehicle access area near the campus entrance.
22	Open Recreation Ground	Recreation	Campus space for recreation and outdoor activities.

7. CAMPUS GRAPH DATASET: PATHWAYS

paths.txt contains one record per undirected connection in the format fromId|toId|distance. The inspected file has 31 records. All records are initially treated as open unless a matching blockage is loaded from path_status.txt.

IDs	Connected locations	Distance (m)	Initial state
1 – 21	Main Gate – Main Parking Area	80	OPEN*
1 – 5	Main Gate – Administrative Block	140	OPEN*
1 – 2	Main Gate – Academic Block	220	OPEN*
21 – 5	Main Parking Area – Administrative Block	100	OPEN*
5 – 2	Administrative Block – Academic Block	120	OPEN*
2 – 3	Academic Block – Central Library	90	OPEN*
2 – 4	Academic Block – Laboratory Complex	110	OPEN*
3 – 4	Central Library – Laboratory Complex	75	OPEN*
3 – 6	Central Library – Auditorium	160	OPEN*
4 – 8	Laboratory Complex – College Canteen	180	OPEN*
5 – 7	Administrative Block – Guest House	130	OPEN*
7 – 10	Guest House – Shopping Area	100	OPEN*
6 – 8	Auditorium – College Canteen	95	OPEN*
6 – 9	Auditorium – Sports Complex	170	OPEN*
8 – 10	College Canteen – Shopping Area	85	OPEN*
8 – 22	College Canteen – Open Recreation Ground	120	OPEN*
9 – 22	Sports Complex – Open Recreation Ground	90	OPEN*
10 – 11	Shopping Area – Hostel D	140	OPEN*

IDs	Connected locations	Distance (m)	Initial state
11 – 12	Hostel D – Hostel E	45	OPEN*
12 – 13	Hostel E – Hostel F	50	OPEN*
13 – 14	Hostel F – Hostel G	45	OPEN*
14 – 15	Hostel G – Hostel I	70	OPEN*
15 – 16	Hostel I – Hostel J	50	OPEN*
16 – 17	Hostel J – Hostel K	160	OPEN*
18 – 19	Hostel L – Hostel P	65	OPEN*
19 – 20	Hostel P – Hostel Q	45	OPEN*
20 – 9	Hostel Q – Sports Complex	190	OPEN*
14 – 22	Hostel G – Open Recreation Ground	150	OPEN*
11 – 8	Hostel D – College Canteen	170	OPEN*
17 – 10	Hostel K – Shopping Area	130	OPEN*
17 – 18	Hostel K – Hostel L	45	OPEN*

*OPEN indicates the default Path constructor state in the absence of a persisted matching closure. The supplied path_status.txt file is empty.

Graph representation The implementation stores the edge set as a vector<Path> and checks incidence with Path::getOtherEnd(). The source does not construct an adjacency-list or adjacency-matrix data structure, despite the earlier PDF's adjacency-representation wording.

8. ROUTE OPTIMIZATION USING DIJKSTRA'S ALGORITHM

8.1 Cost model

Dijkstra's algorithm requires non-negative edge costs. The project uses positive finite pathway distances. In shortest-distance mode, the cost of an edge is its distance in metres. In fastest-estimated-time mode, the cost is distance divided by an assumed walking speed:

```
speed(d) = 1.2 m/s, when d < 150 m
           = 1.5 m/s, when d >= 150 m
time_cost(d) = d / speed(d) seconds
```

8.2 Algorithm steps

- Initialize every tentative cost to infinity; set the source cost to zero.
- Repeatedly scan all configured locations and choose the unvisited location with the smallest tentative cost.
- Stop if no reachable unvisited location remains or the destination is selected.
- Scan the Path vector; skip blocked paths and paths not incident to the current location.
- Calculate the edge cost for the selected mode and relax the neighbour if the new cost is smaller.
- Store the predecessor ID, then reconstruct the route backwards from destination and reverse it.

8.3 Verified implementation excerpt

```
if (path.isBlocked()) continue;
int neighbour = path.getOtherEnd(current);
if (neighbour == -1 || visited[neighbour]) continue;

double edgeCost = fastest
    ? path.getDistance() / assumedSpeed
    : path.getDistance();

double newDistance = distance[current] + edgeCost;
if (newDistance < distance[neighbour]) {
    distance[neighbour] = newDistance;
    previous[neighbour] = current;
}
```

8.4 Output interpretation

The selected route is printed as a sequence of location names. The program then independently sums the physical distances and estimated walking times along that route. Estimated time is always displayed, including when shortest-distance mode selected the route. It is an approximation based on the same distance-threshold speed assumptions, not a measured travel time.

9. PATH MANAGEMENT AND DYNAMIC RE-ROUTING

9.1 Administrative operations

Operation	Implementation behaviour
Add	Checks distinct valid endpoints, finite positive distance and duplicate connection; appends then saves paths.txt.
Edit distance	Requires a finite positive distance; updates matching connection in either endpoint order and saves.
Remove	Erases matching connection, saves paths.txt and rewrites closure records; attempts rollback if saving fails.
Block	Sets blocked=true and stores a reason; writes path_status.txt. If save fails, clears the in-memory blocked state.
Restore	Sets blocked=false and clears reason; rewrites path_status.txt. If save fails, restores previous state and reason.

9.2 Re-routing example: Hostel E to Open Recreation Ground

The original report describes blocking the Hostel F–Hostel G edge (IDs 13–14). The graph supports the following distance calculations:

Scenario	Route	Distance
All paths open	12 Hostel E → 13 Hostel F → 14 Hostel G → 22 Open Recreation Ground	50 + 45 + 150 = 245 m
13–14 blocked	12 Hostel E → 11 Hostel D → 8 College Canteen → 22 Open Recreation Ground	45 + 170 + 120 = 335 m

Estimated time for the first route using the code's speed rule is 50/1.2 + 45/1.2 + 150/1.5 = 179.17 seconds, or about 2.99 minutes. This agrees with the original PDF's approximately 2.98-minute figure to rounding/truncation. The second route is 37.5 + 113.33 + 100 = 250.83 seconds, or about 4.18 minutes, consistent with the PDF's rounded value.

Interpretation This is a worked calculation from the supplied graph and routing rule. The ZIP's path_status.txt is empty, and no recorded console transcript is included; therefore the example is not presented as an archived test log.

10. FILE PERSISTENCE AND PROGRAM WORKFLOW

10.1 File formats

File	Record format	Purpose
paths.txt	fromId|toId|distance	Stores the current set of undirected connections and distances.
path_status.txt	fromId|toId|reason	Stores blocked connections and their closure reasons; only blocked paths are written.

10.2 Loading and saving

loadPaths() parses each non-empty line, requires exactly two separators, converts the fields, checks trailing characters, validates endpoint IDs and distance, and rejects duplicate undirected edges. It builds a temporary vector and swaps it into paths only after the whole file has been read successfully. loadPathStatuses() parses blockage records and applies them to matching paths; malformed records are ignored.

savePaths() rewrites the path list. savePathStatuses() rewrites only blocked records. These are plain text files in the process working directory, so the executable must be run from a directory where the expected files are present.

10.3 Main program workflow

```
Startup → Main menu
  1 Map | 2 Directory | 3 Search | 4 Statistics
  5 Route → source ID, destination ID, mode
  6 View paths
  7 Admin login → path management submenu
  0 Exit
```

10.4 Input validation and admin gate

readInt() reads a line, uses std::stoi(), accepts trailing whitespace only, and retries on malformed input. The distance prompts separately check stream extraction. Closure reasons cannot be blank. The administrator gate compares credentials against literals in main.cpp (a username and password stored as source-code literals); this is a demonstration-level gate, not secure authentication. Credentials should be changed before any public deployment.

11. IMPLEMENTATION, TESTING AND COMPLEXITY

11.1 Source layout and build

```
SmartCampusNav/
  main.cpp
  paths.txt
  path_status.txt
  src/
    Campus.h   Campus.cpp
    Location.h Location.cpp
    Path.h     Path.cpp
```

The project uses standard C++ headers and C++17-compatible features. From the project root, a GCC/MinGW build command is:

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp \
  src/Location.cpp src/Path.cpp src/Campus.cpp \
  -o SmartCampusNav
```

Run the resulting executable from the project root so that paths.txt and path_status.txt resolve as expected.

11.2 Testing evidence and limits

The source archive contains no test source files or automated test harness in the tests/ directory (the directory is empty), and no captured terminal test transcript. During preparation of this report, the supplied source was compiled successfully with the stated GCC C++17 command (no compiler diagnostics). Controlled console runs then confirmed: (a) Hostel E (12) to Open Recreation Ground (22) produced a 245 m route with approximately 2.986 minutes estimated time; (b) blocking Hostel F–Hostel G (13–14) produced the 335 m alternative via Hostel D and College Canteen, with approximately 4.181 minutes estimated time; and (c) after process restart, the blockage remained in effect. These are limited smoke checks, not a comprehensive test suite. The earlier PDF's PASSED matrix is not independently reproducible as a packaged test report.

Scenario	Expected behaviour from code	Evidence status
Startup	Loads fixed 22-location catalogue and parses 31 edge records.	Compiled and run; 31 paths loaded
Search	ID lookup and case-insensitive substring search.	Code inspected; not separately run
Route modes	Dijkstra with distance or time edge costs.	Console run: 245 m; 2.986 min
Block and reroute	Blocked edge skipped by route solver.	Console run: 335 m alternative
Persistence	Saved blockage reloaded on next startup.	Restart run retained closure
Invalid path data	Malformed, duplicate, invalid endpoint or non-positive distance rejected.	Code inspected; not separately run

11.3 Complexity analysis

Let V be the number of locations and E the number of Path objects. The solver selects the next vertex by scanning all locations, costing O(V) per iteration and $O(V^2)$ over at most V iterations. For each selected vertex it scans the entire paths vector, costing O(E) per iteration and O(VE) overall. Thus the implementation's route calculation is $O(V^2 + VE)$, with O(V) auxiliary arrays for tentative

costs, predecessors and visited flags, plus O(V) route storage. This is more precise than the $O(V^2)$ estimate in the earlier PDF because the edge vector is scanned in each iteration.

Directory display is O(V); ID lookup and name search are O(V) (excluding string-comparison length). Path listing and path management generally scan O(E). Loading/saving path data is O(E) in the number of records, with duplicate checking during loading adding $O(E^2)$ worst-case work in the current nested scan.

12. LIMITATIONS, FUTURE WORK AND CONCLUSION

12.1 Current limitations

- Terminal-only interface; the “map” is a fixed logical text schematic, not an interactive or geographically scaled map.
- Campus coordinates and distances are logical data supplied in a text file; no GPS survey or location detection is implemented.
- Travel time is estimated with two fixed speed assumptions and does not account for user pace, crowding, weather, slope or live conditions.
- The administrator credentials are hard-coded in the source and are not suitable for production security.
- Data files are local flat files with no concurrent-user coordination or transactional database.
- The route solver assumes location IDs are contiguous and can index vectors sized locations.size()+1.

12.2 Future enhancements (proposed, not implemented)

- A GUI map could replace the text schematic while preserving the Campus route interface.
- A surveyed coordinate dataset and optional GPS integration could improve spatial accuracy.
- Accessibility metadata could distinguish ramps, stairs and restricted paths, enabling accessible-route constraints.
- Authentication should use secure credential storage and role management rather than source-code literals.
- A database or validated atomic file-update strategy could improve concurrent access and recovery.
- Automated unit and integration tests should capture route costs, blocked-edge behaviour and persistence across restarts.

12.3 Conclusion

The project demonstrates a focused C++ application in which Location and Path model campus entities, Campus coordinates graph operations, and Dijkstra’s algorithm computes a route under two cost criteria. Blocking a Path removes it from consideration during route calculation, while local text files preserve the edge list and closure state. The source is a useful third-semester exercise in encapsulation, composition, standard-library containers, algorithm design, input validation and file streams. Its boundaries—especially the console interface, logical dataset and estimated travel times—are explicit and should be retained when describing the system.

REFERENCES

1. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

3. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.

4. SmartCampusNav source archive supplied for this report: main.cpp; src/Campus.h/.cpp; src/Location.h/.cpp; src/Path.h/.cpp; paths.txt; path_status.txt. (Implementation evidence; local project files.)

5. Smart Campus Navigation System – Academic Project Report (6).pdf. (Prior report used for structure and project context; technical claims reconciled against source code.)

APPENDIX A. VIVA-READY TECHNICAL SUMMARY

Question	Concise answer grounded in this implementation
What is a vertex?	A campus Location identified by an integer ID.
What is an edge?	A Path connecting two location IDs; it is treated as undirected.
Where are edges stored?	In Campus::paths, a std::vector<Path>; not in an adjacency list.
How are blocked edges handled?	findRoute() skips a Path when isBlocked() returns true.
What does fastest mode optimize?	The sum of per-edge estimated times using distance-dependent assumed speeds.
What does Campus own?	Value collections of Location and Path objects in vectors.
What is persisted?	The current path list and blocked-path records/reasons.
Is there inheritance?	No inheritance hierarchy or virtual methods appear in the inspected classes.
Is operator overloading used?	Yes. Location defines friend operator<< for output formatting.
What are key limits?	CLI-only, logical distances, heuristic time, local files and hard-coded admin credentials.

APPENDIX B. SOURCE VERIFICATION NOTES

Inspected archive files: main.cpp; src/Campus.h; src/Campus.cpp; src/Location.h; src/Location.cpp; src/Path.h; src/Path.cpp; paths.txt; path_status.txt; README.md. The source confirms the class fields, public operations, file formats, menu structure, graph dataset, path filtering and Dijkstra implementation described above.

Not present as usable implementation evidence: automated test files, captured runtime test logs, GUI assets, GPS integrations, network services, database schema or real-time traffic feeds. The README advertises several features and OOP concepts that are not implemented in the inspected C++ source; this report does not treat those README claims as implemented functionality.

Important correction log: (i) edge storage is a vector<Path>, not an adjacency list; (ii) route-solver complexity includes an O(VE) edge scan term; (iii) the first rerouting example's estimated time is approximately 2.99 minutes under the source rule, consistent with the original report's rounded figure; (iv) path_status.txt in the supplied archive is empty.

APPENDIX C. CORE C++ SOURCE CODE LISTINGS

This appendix provides key verified source listings for the Smart Campus Navigation and Route Optimization System. The code is modularized across Location, Path, and Campus entities.

C.1 Location Entity (src/Location.h & src/Location.cpp Excerpt)

[Demonstrates: Encapsulation, Constructor Initializer Lists, Operator Overloading (operator<<)]

```
//src/Location.h

#ifndef LOCATION_H
#define LOCATION_H

#include <string>
#include <iostream>

class Location
{
private:
    int id;
    std::string name;
    std::string category;
    std::string description;

public:
    Location(int id, std::string name, std::string category, std::string descrip-
tion);

    int getId() const;
    std::string getName() const;
    std::string getCategory() const;
    std::string getDescription() const;
    void displayDetails() const;

    // Overload the output operator for Location objects.
    friend std::ostream& operator<<(std::ostream& out, const Location& location);
};

#endif
```

```cpp
//src/Location.cpp

#include "Location.h"
#include <iostream>
#include <ostream>
#include <utility>

Location::Location(int id, std::string name, std::string category, std::string
description)
    : id(id), name(std::move(name)), category(std::move(category)), descrip-
tion(std::move(description))
{
}

int Location::getId() const { return id; }
std::string Location::getName() const { return name; }
std::string Location::getCategory() const { return category; }
std::string Location::getDescription() const { return description; }

std::ostream& operator<<(std::ostream& out, const Location& location)
{
    out << "\n-------------------------------------------------------------\n"
        << " Location ID   : " << location.id << "\n"
        << " Name          : " << location.name << "\n"
        << " Category      : " << location.category << "\n"
        << " Description   : " << location.description << "\n"
        << "-------------------------------------------------------------\n";

    return out;
}

void Location::displayDetails() const
{
    std::cout << *this;
}
```

C.2 Path Entity (src/Path.h & src/Path.cpp Excerpt)

[Demonstrates: Encapsulation, Undirected Edge Traversal (getOtherEnd)]

```
//src/Path.h

#ifndef PATH_H
#define PATH_H

#include <string>

class Path
{
private:
    int startId;
    int endId;
    double distance;
    bool blocked;
    std::string blockReason;

public:
    Path(int startId, int endId, double distance, bool blocked = false);

    int getStartId() const;
    int getEndId() const;
    double getDistance() const;
    void setDistance(double newDistance);

    bool isBlocked() const;
    void setBlocked(bool status, const std::string& reason = "");
    std::string getBlockReason() const;
    int getOtherEnd(int locationId) const;
};

#endif
```

```
//src/Path.cpp

#include "Path.h"

Path::Path(int startId, int endId, double distance, bool blocked)
    : startId(startId), endId(endId), distance(distance), blocked(blocked)
{
```

```
}

int Path::getStartId() const { return startId; }
int Path::getEndId() const { return endId; }
double Path::getDistance() const { return distance; }
void Path::setDistance(double newDistance)
{
    distance = newDistance;
}
bool Path::isBlocked() const { return blocked; }
void Path::setBlocked(bool status, const std::string& reason)
{
    blocked = status;
    blockReason = status ? reason : "";
}
std::string Path::getBlockReason() const { return blockReason; }

// Since paths are bidirectional, return the opposite endpoint of this path.
int Path::getOtherEnd(int locationId) const
{
    if (locationId == startId) return endId;
    if (locationId == endId) return startId;
    return -1; // This path is not connected to the supplied location.
}
```

C.3 Campus Controller Header (src/Campus.h)

[Demonstrates: Composition (std::vector<Location> & std::vector<Path>), Public Abstraction Layers]

```
//src/Campus.h

#ifndef CAMPUS_H
#define CAMPUS_H

#include <string>
#include <vector>
#include "Location.h"
#include "Path.h"

class Campus
{
private:
    std::vector<Location> locations;
    std::vector<Path> paths;
```

```
    void loadLocations();
    void loadPaths();
    void loadPathStatuses();

    bool savePaths() const;
    bool savePathStatuses() const;

    const Location* findLocation(int id) const;
    static std::string toLower(std::string text);

public:
    Campus();

    void displayLocations() const;
    void searchByName(std::string keyword) const;
    void searchById(int id) const;
    void displayMap() const;
    void displayStatistics() const;

    bool hasBlockedPaths() const;

    void displayPaths(bool showBlockedOnly = false,
                      bool showOpenOnly = false) const;

    bool addPath(int fromId, int toId, double distance);
    bool editPathDistance(int fromId, int toId, double newDistance);
    bool removePath(int fromId, int toId);

    bool blockPath(int fromId, int toId, const std::string& reason);
    bool restorePath(int fromId, int toId);

    void findRoute(int sourceId, int destinationId, bool fastest = false) const;
};

#endif
```

C.4 Dijkstra Pathfinding Engine Excerpt (src/Campus.cpp - findRoute())

[Demonstrates: Graph Algorithm, Dynamic Edge Filtering (isBlocked), Speed Heuristics]

```
//src/Campus.cpp
//NOTE- includes just the core implementation of the oop concepts, not the actual
entire file as in the github repository.

#include "Campus.h"
#include <iostream>
#include <vector>
#include <limits>
#include <algorithm>

// Constructor coordinates the 3-step startup sequence
Campus::Campus() {
    loadLocations();
    loadPaths();
    loadPathStatuses();
}

// Core Pathfinding Engine: Dijkstra's SSSP with Dynamic Edge Filtering
void Campus::findRoute(int sourceId, int destinationId, bool fastest) const {
    if (!findLocation(sourceId) || !findLocation(destinationId)) {
        std::cout << "\nInvalid source or destination ID.\n";
        return;
    }

    const double INF = std::numeric_limits<double>::infinity();
    std::vector<double> distance(locations.size() + 1, INF);
    std::vector<int> previous(locations.size() + 1, -1);
    std::vector<bool> visited(locations.size() + 1, false);

    distance[sourceId] = 0.0;

    for (std::size_t count = 0; count < locations.size(); ++count) {
        int current = -1;
        double bestDistance = INF;

        // Greedy selection of unvisited vertex with minimum tentative cost
        for (const Location& location : locations) {
            int id = location.getId();
            if (!visited[id] && distance[id] < bestDistance) {
                bestDistance = distance[id];
                current = id;
            }
```

```
        }

        if (current == -1 || current == destinationId) break;
        visited[current] = true;

        // Edge relaxation and dynamic blockage check
        for (const Path& path : paths) {
            if (path.isBlocked()) continue; // Skip temporary maintenance clo-
sures

            int neighbour = path.getOtherEnd(current);
            if (neighbour == -1 || visited[neighbour]) continue;

            // Distance vs. Fastest time heuristic
            double assumedSpeed = (path.getDistance() >= 150.0) ? 1.5 : 1.2;
            double edgeCost = fastest ? (path.getDistance() / assumedSpeed) :
path.getDistance();

            if (distance[current] + edgeCost < distance[neighbour]) {
                distance[neighbour] = distance[current] + edgeCost;
                previous[neighbour] = current;
            }
        }
    }

    // Route reconstruction and terminal formatting logic follows...
}
// Administrative Path Blocking with State Persistence
bool Campus::blockPath(int fromId, int toId, const std::string& reason) {
    for (Path& path : paths) {
        bool match = (path.getStartId() == fromId && path.getEndId() == toId) ||
                     (path.getStartId() == toId && path.getEndId() == fromId);
        if (match) {
            if (path.isBlocked()) return false;
            path.setBlocked(true, reason);

            if (!savePathStatuses()) { // Transactional safety rollback
                path.setBlocked(false);
                return false;
            }
            return true;
        }
    }
    return false;
}
```